Universal Turing Machine (UTM)

→ UTM is a Theoretical concept proposed by Allan Turing. 1930

→ UTM is a Device capable of Executing any computation just like general purpose computation.

Suppose a computer perform 10 operation then UTM is capable of all 10 operation

→ A Turing machine is a mathematical model for doing any computational problem.

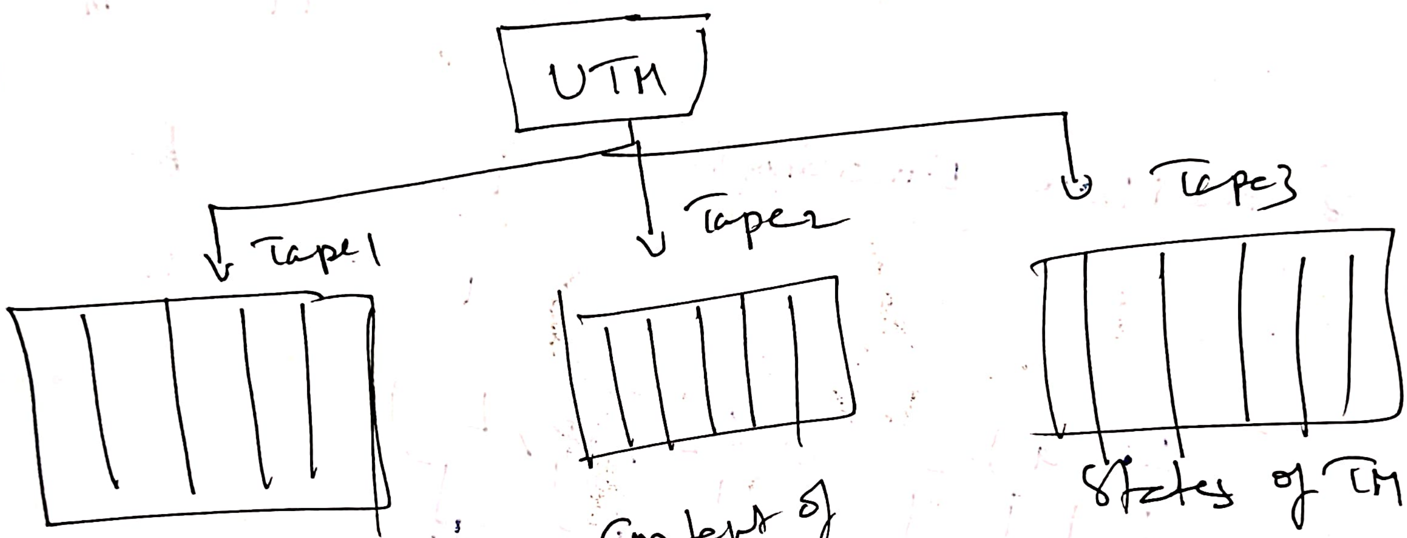
→ We have one TM for addition
second TM for Multiplication
Third TM for substraction
- -

But ~~(50)~~ with the help of UTM we can perform all the operations in one ~~the~~ TM.

Imp → A TM is said to be UTM, if it can simulate the behaviour of any TM

→ UTM can be used to solve any computational problem.

→ Standard TM is "Unprogrammable TM" as it works ~~on~~ for only 1 computational model.. However UTM is "programmable TM" as it works for all the computational problems



Description of TM

Content of TM

States of TM

Ex Let us Assume we are performing Even palindromes then the Description of Palindrome is stored in Tape 1

input content (or) output content

on which state currently we are performing the operation

We describe TM as a string of symbol.

Ex ① **Alphabet Encoding** :-

a	b	c
$\downarrow$	$\downarrow$	$\downarrow$
1	11	111

② **State Encoding** :-

q_1	q_2	q_3
1	11	111

③ Head Move **Encoding** :-

L	R
1	11

$$\delta(q_1, a) = (q_2, b, L)$$

1 0 1 0 11 0 11 0 1

P, NP, NP-Hard and NP-Complete Problems

Based on Time Complexity The Algorithms are classified into two types.

① **Polynomial Time Algorithms :-** (Time Complexity is Less)

Ex:- ① Linear Search $O(n)$

② Bubble sort $O(n^2)$

③ Merge sort $O(n \log n)$

etc..

② **Non polynomial or Exponential Algorithms :-**
(Time Complexity is very high)

Ex:- ① Travelling salesman problem $O(n^2 \cdot 2^n)$

② Knapsack problem $O(\frac{2^n}{2})$

→ Based on the results the problems are Divided into Two types.

① **Decision Problem**

The Result of this problem is Yes or No
(or) $\{0,1\}$

Ex Linear search :- It will give the key Element is found or Not found.

②

(2) Optimization problems.

The Result of this problem is a number representing an object value.

P-class problem

P is set of problems that can be solved in Polynomial time by using deterministic Algorithm. i.e All the steps in the Algorithm is easily ~~understa~~ Understandable. Those Algorithms are called Deterministic Algorithms.

Ex:- Linear search $O(n)$, Binary Search $O(\log n)$ etc.

Formally, an algorithm is polynomial time Algorithm if there exist a polynomial $P(n)$ such that the algorithm can solve any instance of size 'n' in a time $O(P(n))$.

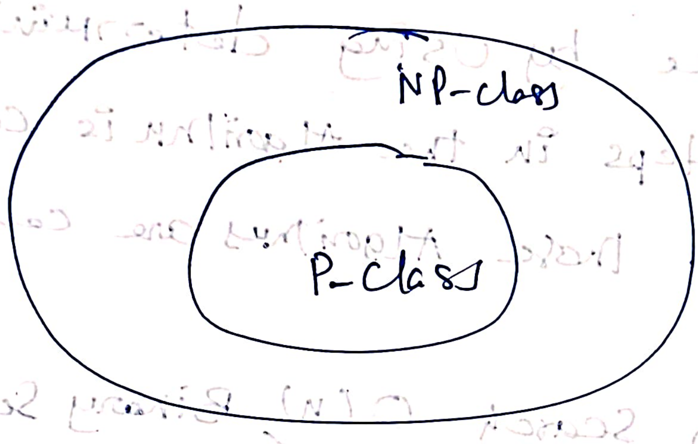
NP- class Problems

NP is set of Problems that can be solved in Exponential time by using Non Deterministic Algorithms But this kind of problems can be verified in Polynomial Time.

Ex:- Travelling Sales Man Problem.

To solve the Travelling Salesman problem. Its running time is very high. But we can verify this problem by using polynomial time by using Shortest path Algorithm.

Difference Between P and NP-Problems



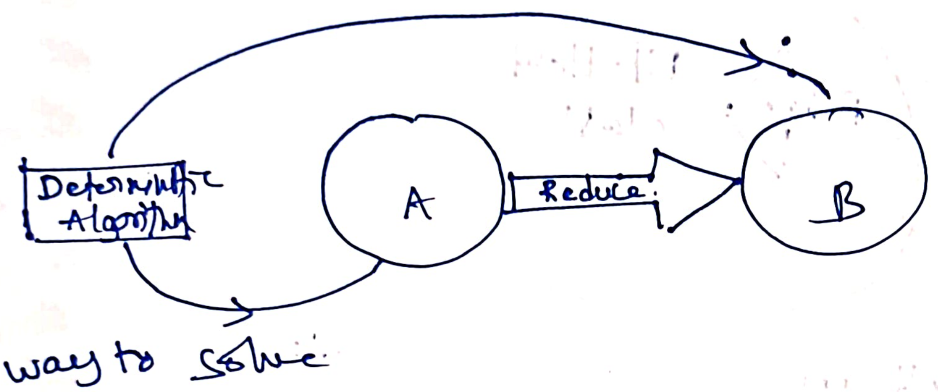
P-class	NP-class
(1) Solved in Polynomial Time	(1) Solved in Non-Polynomial Time
(2) Verified in polynomial Time.	(2) Verified in polynomial Time
(3) Tractable problem	(3) Tractable problem

Note: It is not known whether (1) P = NP
(2) P ≠ NP

Reduction :- let A and B are NP problems.

If problem A is reduce to problem B, iff there is a way to solve A by deterministic Algorithm that solve B in polynomial time.

Then we can denot A $\propto$ B

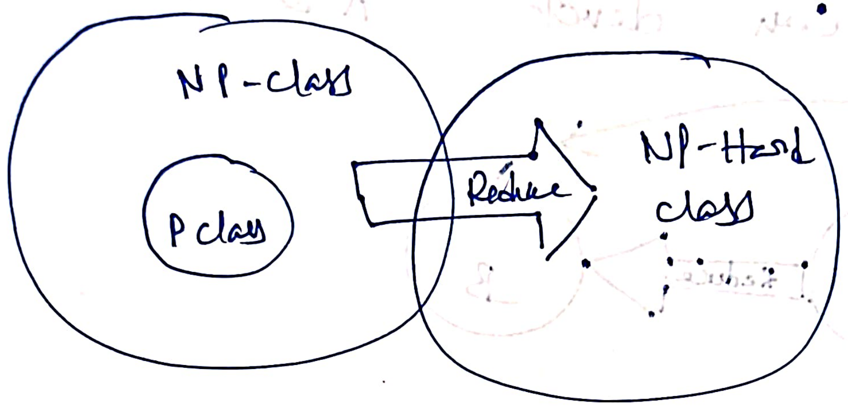


Properities :- ① If A is reducible to B and B is solved in Polynomial time, then A is also solved in polynomial time.

② If 'A' is not in polynomial Time, it implies that 'B' is not in polynomial Time.

NP- Hard Problem

Every Problem in NP Class can be reduced into other set using polynomial time, then it is called as NP- Hard problem.



NP- complete Problems

The group of problems which are both is NP and NP- Hard are known as NP-Complete problem.

→ All NP- complete problems are NP- Hard problem but not all NP- Hard problems are not NP- Complete problem.